Darwin
A Privacy-Focused Research Agent
Built on the Model Context Protocol

Mohammad Ammar **Divya Vinaybhai Patel** **Samantha Susan John**

April 2026

Abstract. Darwin is a fully local, privacy-preserving research assistant that connects a large language model running on Ollama to external academic tools through the Model Context Protocol (MCP). The system integrates an ArXiv MCP server for paper discovery and retrieval, a PDF parsing server for content extraction, and an Obsidian server for persistent knowledge management. A multi-turn agentic control loop drives autonomous literature review workflows, while a human-in-the-loop approval mechanism, local file caching, structured logging, and robust error handling make the pipeline production-ready. All computation and storage remain on the user's machine, with no data sent to external services beyond the arXiv API.

Contents

1 Introduction **2**

2 System Architecture **2**
- 2.1 Overview . . . 2
- 2.2 Layer Breakdown . . . 3
  - 2.2.1 Agent Orchestrator Layer . . . 3
  - 2.2.2 Model Client Layer . . . 3
  - 2.2.3 MCP Server Layer . . . 3

3 Implementation **4**
- 3.1 ArXiv MCP Server . . . 4
  - 3.1.1 Local Caching . . . 4
  - 3.1.2 Logging . . . 4
- 3.2 PDF Parser MCP Server . . . 5
- 3.3 Agent Orchestrator . . . 5
  - 3.3.1 Multi-Turn Tool Execution Loop . . . 5
  - 3.3.2 Argument Type Coercion . . . 5
  - 3.3.3 OllamaClient . . . 6
  - 3.3.4 MCPClient . . . 6
- 3.4 Human-in-the-Loop Approval . . . 6
- 3.5 System Prompt & Agent Persona . . . 6
- 3.6 Streamlit User Interface . . . 6

4 End-to-End Workflow **7**

5 Feature Summary **8**

6 Privacy & Design Properties **8**

7 Conclusion **9**

1. Introduction

Literature review is one of the most time-intensive steps in any research workflow. Researchers must search across databases, read and evaluate dozens of papers, extract relevant findings, and maintain structured notes — all while ensuring that sensitive research topics and unpublished ideas are not unnecessarily exposed to cloud services.

Darwin addresses these challenges through a single, unified agent that runs entirely on the user's local machine. The system is built on three design principles:

1. **Privacy by default.** Papers are downloaded locally, PDFs are parsed locally, and the language model runs on local hardware via Ollama. No research queries or document contents are sent to external cloud inference endpoints.

2. **Modularity through MCP.** The Model Context Protocol provides a standard interface between the language model and external tool servers. Each capability (arXiv search, PDF parsing, Obsidian notes) is an independent server that can be upgraded or replaced without modifying the orchestrator.

3. **Agentic autonomy with human oversight.** The agent executes multi-step workflows autonomously but enforces a human-in-the-loop gate before downloading papers, ensuring the user retains control over what enters their local storage.

The result is a system that can perform a full literature review — search, filter, download, parse, summarise, and archive — within a single conversational session, on a standard laptop, with no cloud dependencies beyond the public arXiv API.

2. System Architecture

2.1 Overview

Darwin follows a layered, modular architecture. The three layers are kept strictly separate so that each can be tested and extended independently.

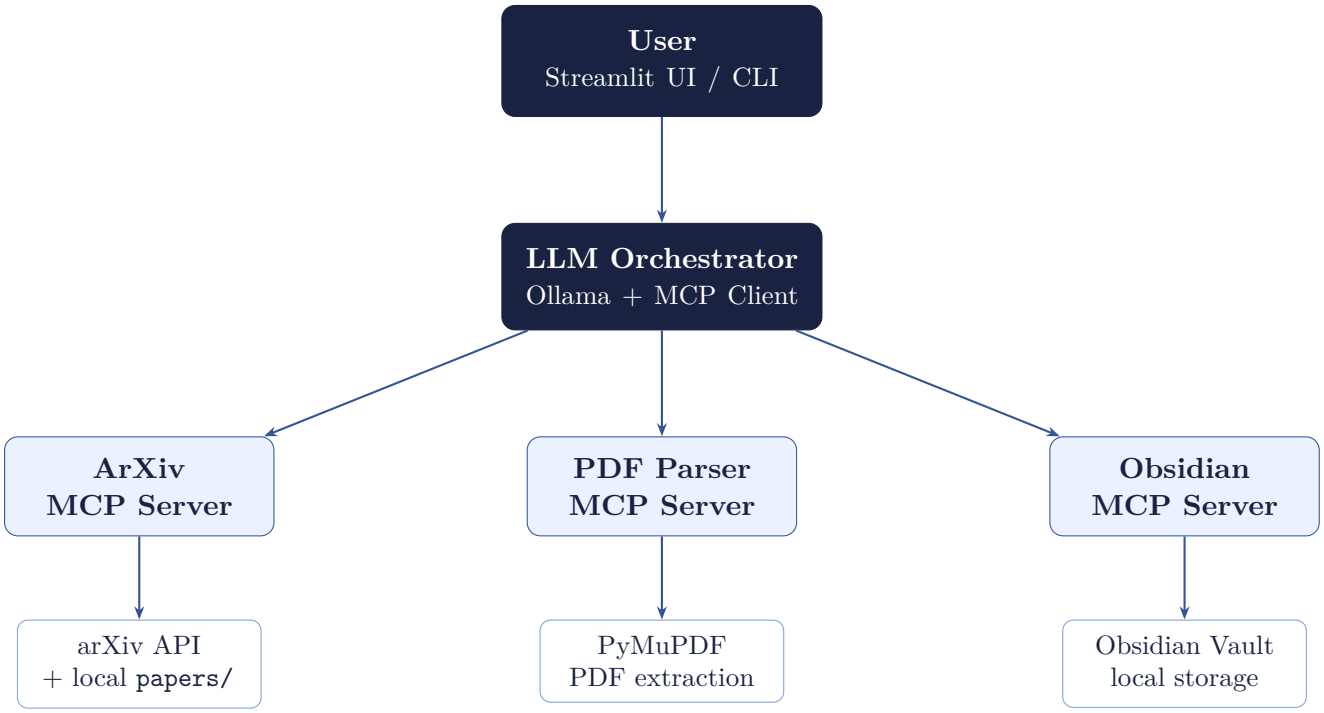

Figure 1: High-level architecture of the Darwin system.

2.2 Layer Breakdown

Agent Orchestrator Layer

The orchestrator (`src/agent/`) is the central reasoning engine. It initialises the Ollama language model client (`OllamaClient`) and the MCP session manager (`MCPClient`), then enters a multi-turn conversation loop. On each turn it:

- Sends the current message history and available tool schemas to the local LLM.
- Parses the model response for tool calls.
- Executes approved tool calls against the appropriate MCP server.
- Appends tool results to message history and re-invokes the model.
- Repeats until the model produces a final response with no further tool requests.

Model Client Layer

`OllamaClient` wraps the Ollama Python SDK. It accepts a model name, host URL, and optional system prompt at construction time and exposes a single `chat()` method that forwards messages and tool schemas to the Ollama `/api/chat` endpoint with `stream=False`. Switching the underlying model requires only a one-line config change.

MCP Server Layer

`MCPClient` manages connections to all configured MCP servers simultaneously using an `AsyncExitStack`. At startup it reads `config/claude_desktop_config.json`, opens a stdio transport session for each server, normalises their tool schemas into OpenAI-compatible function-calling format, and presents the merged tool list to the orchestrator.

3. Implementation

3.1 ArXiv MCP Server

The ArXiv server (`src/arxiv_server/server.py`) is built with FastMCP and exposes six tools covering the full paper lifecycle:

Tool	Description
`search_papers`	Queries the arXiv API with a keyword string and a configurable result cap. Returns structured metadata: ID, title, publication date, summary, and PDF URL.
`download_paper`	Downloads a paper by arXiv ID to the local `papers/` directory. Checks for an existing cached file before making any network request.
`list_papers`	Returns a directory listing of all locally cached PDFs.
`read_paper`	Opens a local PDF with PyMuPDF and returns full page text. Bounded by a configurable character cap to protect the model's context window.
`confirm_download`	Formats paper metadata into a human-readable confirmation request and returns it to the orchestrator, which presents it to the user before any download is initiated.
`log_research_action`	Persists a timestamped action record (operation type, paper ID, result payload) to a local SQLite database (`research_log.db`).

Listing 1: ArXiv MCP server startup and tool discovery

```
[INFO] Starting ArXiv MCP Server...
Connected to arxiv
Discovering tools...
Found 8 tools.
  - search_papers
  - download_paper
  - list_papers
  - read_paper
  - confirm_download
  - log_research_action
  - extract_pdf_sections
  - extract_figures
Agent ready. Type 'exit' to quit.
```

Local Caching

Before every download the server iterates `papers/` and returns the existing path if a file with a matching arXiv ID prefix is found. This eliminates redundant network calls across sessions and makes repeated experiments fully deterministic.

Logging

Every agent action is recorded via `log_research_action`. The SQLite schema is:

```
CREATE TABLE IF NOT EXISTS actions (
```

```
    timestamp TEXT,
    action    TEXT,
    paper_id  TEXT,
    result    TEXT
)
```

This creates an auditable trail that can be exported for debugging or post-run quality evaluation.

3.2 PDF Parser MCP Server

The PDF parser (`src/pdf_parser/server.py`) is a separate FastMCP server providing two tools:

- `extract_pdf_sections` — opens a PDF with PyMuPDF and returns the full concatenated page text for downstream section-level analysis by the LLM.
- `extract_figures` — provides the interface for figure extraction (extensible for richer image output in future iterations).

Keeping this as a distinct server preserves a clean service boundary between the paper retrieval flow (arXiv server) and the document intelligence flow (PDF parser), allowing either to be swapped independently.

3.3 Agent Orchestrator

Multi-Turn Tool Execution Loop

The core of Darwin's intelligence is the tool execution loop in `src/agent/main.py`. The loop is structured as a nested `while True` that continues invoking the model until it produces a response with no tool calls:

```
while True:                         # outer: user turns
    user_input = input("You: ")
    messages.append({"role": "user", "content": user_input})

    while True:                     # inner: agent reasoning
        response = ollama.chat(messages, tools=tools)
        tool_calls = response["message"].get("tool_calls", [])

        if not tool_calls:
            break                   # model finished reasoning

        for tc in tool_calls:
            result = await mcp_client.call_tool(
                tc["function"]["name"],
                tc["function"]["arguments"]
            )
            messages.append({"role": "tool", "content": str(result)})
```

This pattern supports compound tasks — such as searching, selecting, downloading, and summarising several papers — within a single user prompt, without requiring the user to manually chain commands.

Argument Type Coercion

The orchestrator includes a `convert_tool_arguments()` function that inspects each tool's JSON schema and casts string arguments to the correct Python type (integer, float) before passing them to MCP servers. This prevents type errors when the LLM serialises numeric arguments as strings.

OllamaClient

`OllamaClient` passes the complete message history and the full tool schema list to the Ollama API on every turn. Tool schemas are already in OpenAI function-calling format (normalised by `MCPClient`), so Ollama can natively select and invoke tools without any prompt engineering.

MCPClient

`MCPClient` opens a separate `stdio` transport session per configured server and manages them all through a single `AsyncExitStack`. Tool lookup resolves the correct server at call time by iterating active sessions, ensuring the orchestrator remains server-agnostic.

3.4 Human-in-the-Loop Approval

Downloads are gated by a two-stage enforcement mechanism:

1. **Workflow gate.** The agent's system prompt instructs the model to always call `confirm_download` before `download_paper` for any keyword-driven search flow. The `confirm_download` tool returns formatted paper metadata (title, ID, date, abstract preview) that the orchestrator displays to the user before requesting a `yes/no/skip` decision.

2. **Safety net.** Even if the model skips the workflow gate, the orchestrator's `download_paper` handler checks an `approved_papers` set and blocks any unapproved ID with an explicit `BLOCKED` message fed back to the model.

Users can opt out of the gate per-query by including the phrases `without approval` or `auto-download` in their message, or by providing a direct arXiv ID (e.g. `download 2306.04338v1`).

3.5 System Prompt & Agent Persona

The agent's behaviour is governed by the system prompt in `config/agent_config.json`:

```
"You are Darwin, an AI research assistant.
 You help users search, download, and organize academic papers.

 Rules:
 - ALWAYS call tools to execute actions, never just describe them.
 - Direct paper ID (e.g. 'download 2306.04338v1'):
   call download_paper() immediately, no confirmation needed.
 - Keywords 'without approval' or 'auto-download':
   skip confirmation, download directly.
 - 'search and download' with no keywords:
   call confirm_download() first, then download_paper() if approved.
 - After search_papers returns results, list ALL papers with
   their ID, title, and a short summary. Never list only one."
```

This prompt locks Darwin into an action-first research persona, prevents the common failure mode of the model describing tool calls without executing them, and enforces consistent search result presentation.

3.6 Streamlit User Interface

The Streamlit application (`ui_app.py`) provides a graphical front-end that communicates with the agent as a subprocess via `agent_wrapper.py`. The wrapper reads commands from `stdin` and writes tagged protocol lines (`AGENT_RESPONSE:`, `TOOL_EXECUTE:`, `TOOL_BLOCKED:`, `TOOL_CONFIRM:`, `AGENT_END`) to `stdout`. The UI parses these tags to render structured

output, paper cards, and confirmation dialogs without blocking the Streamlit event loop.

4. End-to-End Workflow

A complete Darwin session follows this sequence:

1. **Startup.** The orchestrator loads `agent_config.json`, initialises `OllamaClient` and `MCPClient`, connects to all configured MCP servers, and discovers the merged tool list.
2. **Search.** The user submits a research topic. Darwin calls `search_papers` and presents all results with IDs, titles, and one-line summaries.
3. **Approval.** For each candidate paper the agent calls `confirm_download`, displays the abstract preview, and waits for user confirmation.
4. **Download & cache.** Approved papers are fetched via `download_paper` and saved to `papers/`. Subsequent runs retrieve them from cache without any network call.
5. **Content extraction.** Darwin calls `read_paper` or `extract_pdf_sections` to extract text, then synthesises a per-paper summary.
6. **Knowledge persistence.** The agent calls the Obsidian server to create a structured note for each paper, applying tags by topic, method, and publication year, and linking to the local PDF path.
7. **Logging.** Every action is written to `research_log.db` via `log_research_action` for traceability.
8. **Digest.** The agent produces a final digest note in Obsidian consolidating all findings from the session.

Listing 2: Live agent session searching for interpretability papers

```
You: Find 3 recent papers on mechanistic interpretability in ML,
     then summarise each one.

Executing tool: search_papers...
Args: {'max_results': 3, 'query': 'mechanistic interpretability ML'}
Calling search_papers on arxiv...
[INFO] Found paper: 2306.04338v1
[INFO] Found paper: 1606.05386v1
[INFO] Found paper: 1806.09710v1
[INFO] Returning 3 results

Agent: Found 3 papers:

1. Model-Agnostic Interpretability of Machine Learning (1606.05386v1)
   Proposes a model-agnostic approach to explain ML predictions,
   enabling flexible choice of models and explanations.

2. Changing Data Sources in the Age of Machine Learning (2306.04338v1)
   Highlights risks when changing data sources in ML for official
   statistics --- emphasises robustness and data sourcing integrity.

3. Why Interpretability in Machine Learning? (1806.09710v1)
   Argues for human interpretability, characterises performance using
   distributed detection theory, shows interpretable classifiers
```

```
   outperform black-box ones.

Executing tool: confirm_download...
PAPER DOWNLOAD CONFIRMATION
  Title:     Model-Agnostic Interpretability of ML
  Paper ID:  1606.05386v1
  Published: 2016-06-16
  Abstract:  We propose LIME, a novel explanation technique that
             explains the predictions of any classifier...

Approve? (yes/no/skip): yes
Executing tool: download_paper... -> papers/1606.05386v1.pdf
Executing tool: log_research_action... -> research_log.db
```

5. Feature Summary

Feature	Description	Status
LLM Orchestrator	Multi-turn tool execution loop via Ollama	✓ **Implemented**
ArXiv MCP Server	Search, download, list, and read papers	✓ **Implemented**
PDF Parser Server	Full-text and section extraction via PyMuPDF	✓ **Implemented**
Obsidian Integration	Structured note creation and vault management	✓ **Implemented**
Streamlit UI	Graphical interface with subprocess agent	✓ **Implemented**
Human-in-the-Loop	Two-stage approval gate for paper downloads	✓ **Implemented**
Local Paper Cache	File-based deterministic cache in `papers/`	✓ **Implemented**
Logging & Audit	SQLite action log with full traceability	✓ **Implemented**
Type Coercion	Argument casting from LLM string output	✓ **Implemented**
Error Handling	`try/except` guards on all IO-heavy paths	✓ **Implemented**
System Prompt	Action-first research persona in agent config	✓ **Implemented**
Multi-provider	Swappable model backend via config	✓ **Implemented**

6. Privacy & Design Properties

Data residency. All paper PDFs, extracted text, Obsidian notes, and the action log are stored on the user's local filesystem. The only external communication is the arXiv public API (paper search and PDF download), which does not require authentication and does not receive user identity information.

Model isolation. The language model runs on the local Ollama server (`http://localhost:11434`). No query text, paper content, or conversation history leaves the machine during inference.

Separation of concerns. The three-layer architecture (orchestrator / model client / MCP servers) means that any layer can be audited or replaced independently. Researchers who want to switch from Ollama to a self-hosted API endpoint, or add a new data source, do so without modifying the other layers.

7. Conclusion

Darwin delivers a complete, privacy-preserving research pipeline that operates from a single conversational interface. The combination of a local LLM, a modular MCP server layer, and a multi-turn agentic control loop enables autonomous literature review workflows that would otherwise require manual coordination across multiple tools.

The system is fully operational across all planned phases: the arXiv and PDF servers cover the retrieval and extraction pipeline; the Obsidian integration handles persistent knowledge management; the human-in-the-loop gate and local caching make the agent safe and efficient for repeated use; and the SQLite audit log provides the traceability needed for reproducible research workflows.

Darwin demonstrates that privacy-preserving agentic research assistants are achievable today, without cloud infrastructure, using open-source models and standard protocol primitives. The MCP-based design ensures the system will remain extensible as new data sources and tools become available.

Key insight. The multi-turn tool execution loop in `src/agent/main.py` is the mechanism that transforms Darwin from a static question-answering chatbot into an agentic research workflow engine. Every other feature — caching, approval gates, logging, Obsidian notes — plugs into this loop as independently testable MCP tool calls.